Addis Ababa University — School of Information Technology and Engineering
Software Testing and Validation — Final Project

Test Design Document

Clinic Booking System

Student Biruk Mulatu
Student ID [STUDENT ID — fill in before submission]
Instructor Abel Tadesse
Date 4 September 2026

This document derives test cases for the three domain rules using the four formal techniques the assignment requires: equivalence partitioning, boundary value analysis, a decision table, and state transition testing. Every case listed below is implemented, by the same name, in `backend/src/test/java/et/aau/clinic/unit/` — the map from design to code is exact, not illustrative.

1. Equivalence partitioning and boundary value analysis — Rule 1 (consultation fee by age)

Implemented in `core.FeeCalculator`. The input is a single integer, `age`.

1.1 Partitions

#	Partition	Range	Expected outcome
P1	Invalid — below range	age < 0	Rejected: `IllegalArgumentException`
P2	CHILD	0 ≤ age ≤ 17	Category CHILD, fee 100 ETB
P3	ADULT	18 ≤ age ≤ 64	Category ADULT, fee 250 ETB
P4	SENIOR	65 ≤ age ≤ 120	Category SENIOR, fee 150 ETB
P5	Invalid — above range	age > 120	Rejected: `IllegalArgumentException`

1.2 Boundaries

Three valid partitions sitting side by side produce six interior boundaries (the low and high edge of each), plus the two outer edges where valid meets invalid — eight boundary values in total, exactly the set CLAUDE.md specifies: **–1, 0, 17, 18, 64, 65, 120, 121**.

1.3 Derived test cases

Case	Input (age)	Partition / boundary	Expected	Test method
TC-F01	–1	P1, boundary just below P2	throws	`fee_belowLowerBoundary_ageMinus1_throws`
TC-F02	–50	P1 interior, far from boundary	throws	`fee_wellBelowRange_ageMinus50_throws`
TC-F03	0	P2 lower boundary	CHILD, 100	`fee_atLowerBoundaryOfChildBand_age0_is100`
TC-F04	9	P2 interior	CHILD, 100	`fee_withinChildBand_age9_is100`
TC-F05	17	P2 upper boundary	CHILD, 100	`fee_atUpperBoundaryOfChildBand_age17_is100`
TC-F06	18	P3 lower boundary	ADULT, 250	`fee_atLowerBoundaryOfAdultBand_age18_is250`
TC-F07	40	P3 interior	ADULT, 250	`fee_withinAdultBand_age40_is250`
TC-F08	64	P3 upper boundary	ADULT, 250	`fee_atUpperBoundaryOfAdultBand_age64_is250`
TC-F09	65	P4 lower boundary	SENIOR, 150	`fee_atLowerBoundaryOfSeniorBand_age65_is150`
TC-F10	90	P4 interior	SENIOR, 150	`fee_withinSeniorBand_age90_is150`
TC-F11	120	P4 upper boundary	SENIOR, 150	`fee_atUpperBoundaryOfSeniorBand_age120_is150`
TC-F12	121	P5, boundary just above P4	throws	`fee_aboveUpperBoundary_age121_throws`
TC-F13	200	P5 interior, far from boundary	throws	`fee_wellAboveRange_age200_throws`

13 cases: 3 valid partitions × 3 (low boundary, interior representative, high boundary) = 9, plus 2 invalid partitions × 2 (edge value, far-out interior value) = 4.

2. Decision table — Rule 2 (booking eligibility)

Implemented in `core.BookingPolicy`. Three binary conditions, evaluated in strict priority order C1 → C2 → C3:

- **C1** — the requested slot is still free
- **C2** — the patient has no outstanding balance
- **C3** — the request is made at least 2 hours before the slot start time

2.1 Full table ($2^3 = 8$ rules)

Rule	C1 free	C2 no balance	C3 notice ok	Action
R1	F	–	–	Reject: SLOT_UNAVAILABLE
R2	T	F	–	Reject: OUTSTANDING_BALANCE
R3	T	T	F	Reject: INSUFFICIENT_NOTICE
R4	T	T	T	Approve — appointment created in REQUESTED

Because each condition is checked in priority order with an early return, the naive $2^3 = 8$-row truth table collapses to 4 rules with "don't care" (–) columns for conditions a higher-priority rule already decided — the same shape the implementation itself has (`if (!c1) …; if (!c2) …; if (!c3) …; approve`).

2.2 Derived test cases

For every rule with a don't-care column, two tests are written — one with the don't-care condition true and one false — to prove the outcome genuinely does not depend on it (i.e. that the priority ordering really holds and a lower-priority condition can't override a higher one). Boundary value analysis is then applied separately to the one numeric condition, C3's 2-hour threshold.

Case	C1	C2	C3 / notice given	Expected	Test method
TC-B01	F	T	3h (T)	SLOT_UNAVAILABLE	`decision_slotUnavailable_withGoodBalanceAndNotice_rejectsWithSlotUnavailable`
TC-B02	F	F	30min (F)	SLOT_UNAVAILABLE (proves C1 outranks C2 and C3)	`decision_slotUnavailable_withBadBalanceAndInsufficientNotice_stillRejectsWithSlotUnavailable`
TC-B03	T	F	3h (T)	OUTSTANDING_BALANCE	`decision_outstandingBalance_withSufficientNotice_rejectsWithOutstandingBalance`
TC-B04	T	F	30min (F)	OUTSTANDING_BALANCE (proves C2 outranks C3)	`decision_outstandingBalance_withInsufficientNotice_stillRejectsWithOutstandingBalance`
TC-B05	T	T	30min (F)	INSUFFICIENT_NOTICE	`decision_insufficientNotice_rejectsWithInsufficientNotice`
TC-B06	T	T	3h (T)	Approved	`decision_allConditionsTrue_approves`

2.3 BVA on the C3 threshold (2-hour notice)

Case	Notice given	Boundary	Expected	Test method
TC-B07	exactly 2h00m	lower boundary of "sufficient"	Approved (≥ is inclusive)	`notice_exactlyTwoHoursBeforeSlot_isSufficient_approves`
TC-B08	1h59m (2h – 1min)	just below the boundary	INSUFFICIENT_NOTICE	`notice_oneMinuteLessThanTwoHours_isInsufficient_rejects`
TC-B09	10h	well above the boundary	Approved	`notice_wellOverTwoHours_isSufficient_approves`

3. State transition testing — Rule 3 (appointment lifecycle)

Implemented in `core.AppointmentStateMachine`. States: `REQUESTED, CONFIRMED, ATTENDED, CANCELLED, NO_SHOW`. Events: `CONFIRM, CANCEL, ATTEND, MARK_NO_SHOW`. 5 states × 4 events = 20 pairs.

3.1 State transition table

From \ Event	CONFIRM	CANCEL	ATTEND	MARK_NO_SHOW
REQUESTED	→ CONFIRMED	→ CANCELLED	invalid	invalid

CONFIRMED	invalid	→ CANCELLED	→ ATTENDED	→ NO_SHOW
ATTENDED (terminal)	invalid	invalid	invalid	invalid
CANCELLED (terminal)	invalid	invalid	invalid	invalid
NO_SHOW (terminal)	invalid	invalid	invalid	invalid

5 valid transitions (shown in green), 15 invalid pairs (shown in red) — every event on a terminal state is invalid, and REQUESTED/CONFIRMED each reject the events not listed for them.

3.2 Derived test cases — valid transitions

Case	From	Event	Expected	Test method
TC-S01	REQUESTED	CONFIRM	CONFIRMED	`transition_requestedConfirm_movesToConfirmed`
TC-S02	REQUESTED	CANCEL	CANCELLED	`transition_requestedCancel_movesToCancelled`
TC-S03	CONFIRMED	ATTEND	ATTENDED	`transition_confirmedAttend_movesToAttended`
TC-S04	CONFIRMED	CANCEL	CANCELLED	`transition_confirmedCancel_movesToCancelled`
TC-S05	CONFIRMED	MARK_NO_SHOW	NO_SHOW	`transition_confirmedMarkNoShow_movesToNoShow`

3.3 Derived test cases — invalid transitions

Rather than hand-listing 15 rows (error-prone — easy to miss or duplicate a pair), the test generates *all* 20 state/event pairs and subtracts the 5 valid ones above, then runs a single parameterised test (`transition_invalidPair_throwsIllegalStateException`) once per remaining pair, asserting each throws `IllegalStateException` . A companion test (`invalidStateEventPairs_containsExactlyFifteenPairs`) asserts the generated set has exactly 15 members, so the count itself — not just each individual case — is under test. This guarantees full coverage of the invalid region of the table without the risk of a manually transcribed list silently missing a cell.

3.4 BVA — Rule 3b, late cancellation fee (24-hour guard)

A cancellation of a CONFIRMED appointment charges 50% of the consultation fee if made less than 24 hours before the slot start, and nothing otherwise. This is a second, independent boundary layered on the CANCEL transition.

Case	Time before slot	Boundary	Expected fee (on 250 ETB)	Test method
TC-S06	23h59m	just inside the late window	125.0 (50%)	`lateCancellationFee_atTwentyThreeHours59Minutes_chargesHalfFee`
TC-S07	exactly 24h00m	the boundary itself	0 (free — 24h is not "less than")	`lateCancellationFee_atExactlyTwentyFourHours_isFree`
TC-S08	24h01m	just outside the late window	0 (free)	`lateCancellationFee_atTwentyFourHoursOneMinute_isFree`

4. Summary

Technique	Applied to	Test cases derived
Equivalence partitioning + BVA	Rule 1 (fee by age)	13
Decision table + BVA	Rule 2 (booking eligibility)	9
State transition testing	Rule 3 (lifecycle) — 5 valid + 15 invalid	20 (5 explicit + 15 via one parameterised test + 1 count-check)
BVA	Rule 3b (late cancellation fee)	3
Total, core rules alone		**45 cases** (across the 24 + 9 + 13 = 46 `@Test` /parameterised methods in the three `core/` test classes — the extra method is the 15-pairs count-check itself)

These 45–46 core-level cases are the ones designed directly from formal technique application. The remaining unit tests `AppointmentServiceTest` , 12 methods) and all integration/system tests apply the same rules again through the orchestration and UI layers, and are counted separately in the test summary report's pyramid figures.